

Especificación de requisitos de software

Proyecto: EduTech

[07/04/2025]

Contenido

Ficha del documento	4
1. Introducción	4
1.1. Propósito	4
1.2. Ámbito del Sistema	4
1.3. Definiciones, Acrónimos y Abreviaturas	4
1.4. Referencias	4
1.5. Visión General del Documento	4
2. Descripción General	5
2.1. Perspectiva del Producto	5
2.2. Funciones del Producto	5
2.3. Características de los Usuarios	5
2.4. Restricciones	5
2.5. Suposiciones y Dependencias	6
2.6. Requisitos Futuros	6
3. Requisitos Específicos	7
3.1. Requisitos comunes de los interfaces	7
3.1.1 Interfaces de usuario	7
3.1.2 Interfaces de hardware	7
3.1.3 Interfaces de software	7
3.1.4 Interfaces de comunicación	7
3.2. Requisitos funcionales	8
3.3. Requisitos no funcionales	9
3.3.1 Requisitos de rendimiento	9
3.3.2 Seguridad	9
3.3.3 Fiabilidad	9
3.3.4 Disponibilidad	9
3.3.5 Mantenibilidad	9
3.3.6 Portabilidad	9
3.4. Otros Requisitos	9

Ficha del documento

Fecha	Revisión	Autor	Modificación
20/03/25	25/03/25	Sofía Paillalef Fernanda Manriquez Catrina Corral	Creación de introducción y descripción general del proyecto.
26/03/25	31/04/25	Sofía Paillalef Fernanda Manriquez Catrina Corral	Creación de requisitos específicos, funcionales y no funcionales.
05/04/25	07/04/25	Sofía Paillalef Fernanda Manriquez Catrina Corral	Revisión grupal de documento ERS.

Documento validado por las partes en fecha:

Por el cliente

Por la empresa suministradora

[Firma]

[Firma]

Sr./Sra.

Sr./Sra.

1. Introducción

1.1. Propósito

Este documento define los requisitos funcionales y no funcionales para la migración del sistema monolítico de EduTech Innovators SPA a una arquitectura de microservicios, garantizando escalabilidad, rendimiento y alta disponibilidad.

1.2. Ámbito del Sistema

Nuestro sistema educativo actual utiliza una arquitectura de microservicios para gestionar usuarios, cursos, inscripciones, pagos y comunicaciones entre estudiantes y profesores. Sin embargo, no incluye el desarrollo de aplicaciones móviles nativas, la creación de contenido educativo, la gestión de nóminas, las capacidades avanzadas de inteligencia artificial, la traducción automática o el soporte fuera de línea, **ya que están más allá del alcance** del proyecto actual.

En nuestro proyecto se espera la siguiente:

- **Mayor escalabilidad**
- **Alta disponibilidad**
- **Mejor rendimiento**
- **Seguridad del sistema**

1.3. Definiciones, Acrónimos y Abreviaturas

Microservicio: Se basa en una interfaz de software que funciona de manera autónoma con diversos elementos, los cuales tienen el fin de hacer las mismas tareas. Consta de su propia base de datos, esta se utiliza para hacer aplicaciones.

API Gateway: Es una arquitectura, que actúa como administrador de las peticiones de los usuarios para ingresar a los servicios API.

Kubernetes: Es un programa que es de acceso libre que gestiona aplicaciones a través de contenedores a gran escala.

1.4. Referencias

- Caso de Estudio 2 - EduTech Innovators SPA.

1.5. Visión General del Documento

En este documento, explicaremos cómo modernizar la plataforma educativa EduTech Innovator SPA de un sistema heredado a uno más flexible basado en microservicios.

Este proyecto está destinado a que los equipos técnicos y sus gerentes comprendan mejor el propósito del proyecto, qué esperamos lograr y cómo medimos el éxito del proyecto.

2. Descripción General

Esta sección proporciona información general sobre el nuevo sistema sin entrar en detalles técnicos para que todos comprendan su propósito y alcance. Vamos a ver:

2.1. Perspectiva del Producto

El sistema será una plataforma educativa en línea independiente, pero podrá integrarse con sistemas de pago externos y herramientas de comunicación como foros y chats.

2.2. Funciones del Producto

En esta subsección de la ERS se mostrará un resumen, a grandes rasgos, de las funciones del futuro sistema. Las funciones deberán mostrarse de forma organizada, y pueden utilizarse gráficos, siempre y cuando dichos gráficos reflejen las relaciones entre funciones y no el diseño del sistema.

2.3. Características de los Usuarios

- Administradores: Personal técnico con conocimientos avanzados en gestión de sistemas.
- Gerentes de cursos: Personal con conocimientos en educación y gestión de contenidos.
- Acciones de los clientes vía Web: Usuarios con conocimientos básicos de navegación web.
- Logística de soporte: Personal con habilidades técnicas para resolver incidencias.

2.4. Restricciones

Tecnológicas:

- MySQL para bases de datos.
- Spring Framework para microservicios.
- Infraestructura (Kubernetes y AWS).
- Requisitos de seguridad: Cifrado de datos y autenticación de usuarios.

Diseño:

- Arquitectura: Cada microservicio debe de tener su base de datos independiente.

Operativa:

- Rendimiento.
- Seguridad.
- Cumplimiento legal.
- Criticalidad de la aplicación.
- Consideraciones acerca de la seguridad.

2.5. Suposiciones y Dependencias

- Se asume que los usuarios cuentan con conexión a Internet.
- Se utilizarán navegadores web compatibles (Chrome, Firefox, Edge).
- El sistema dependerá del servidor institucional para su alojamiento.

2.6. Requisitos Futuros

- Plataforma móvil nativa.
- Personalización con IA.
- Herramientas de colaboración.
- Internacionalización.

3. Requisitos Específicos

Esta sección describe los requisitos detallados del sistema, tanto funcionales como no funcionales, interfaces y otros aspectos críticos. Se organiza en subsecciones para facilitar su comprensión y trazabilidad.

3.1 Requisitos comunes de los interfaces

Describe cómo el sistema interactuará con usuarios, hardware, software externo y redes.

3.1.1 Interfaces de usuario

- Web: Interfaz responsive para estudiantes y administradores.
- Móvil: App nativa para iOS y Android (futura implementación).

3.1.2 Interfaces de hardware

- Compatible con estaciones de trabajo estándar.

3.1.3 Interfaces de software

- Autenticación mediante sistema personalizado en Java, utilizando archivos JSON para gestionar roles y permisos.
- Conexión con MySQL como sistema de gestión de base de datos, a través de SQL Developer o herramientas compatibles como DataModeler.

3.1.4 Interfaces de comunicación

- Comunicación básica cliente-servidor usando protocolo HTTP.
- Intercambio de datos mediante archivos JSON.
- La implementación de API REST podrá considerarse en etapas futuras cuando el equipo tenga mayor conocimiento.

3.2 Requisitos funcionales

- **3.2.1 Requisito funcional 1: Registro y autenticación de usuarios.**
 - El sistema debe permitir a los usuarios autenticarse mediante credenciales institucionales.
- **3.2.2 Requisito funcional 2: Gestión de acceso a material didáctico.**
 - El sistema debe permitir que los usuarios accedan al material didáctico en línea, ya sea para visualización directa o descarga. El contenido deberá estar organizado por categorías, curso y autor.
- **3.2.3 Requisito funcional 3: Reportes.**
 - El sistema debe generar reportes de uso e inventario.
- **3.2.4 Requisito funcional 4: Gestión de usuarios (Administrador del sistema).**
 - Crear, actualizar, desactivar y eliminar cuentas de usuario.
 - Asignar y modificar permisos de acceso.
 - Visualizar estado del sistema, alertas de fallos y rendimiento.
 - Respalidar y restaurar datos del sistema.
- **3.2.5 Requisito funcional 5: Gestión de cursos (Gerente de cursos I).**
 - Crear, actualizar y eliminar cursos del catálogo.
 - Generar reportes de inscripción y rendimiento.
 - Asignar instructores a los cursos.
 - Aprobar contenido antes de su publicación.
- **3.2.6 Requisito funcional 6: Gestión de evaluaciones y contenido (Gerente de cursos II).**
 - Desarrollar y subir material didáctico.
 - Crear y actualizar evaluaciones.
 - Corregir pruebas.
 - Guiar a estudiantes mediante foros y chats.
 - Monitorear progreso de estudiantes inscritos.
- **3.2.7 Requisito funcional 7: Soporte (Logística de soporte).**
 - Recibir y resolver incidencias técnicas.
 - Asignar y actualizar estado de incidencias.
 - Optimizar recursos técnicos.
 - Gestionar información de proveedores de servicios.
- **3.2.8 Requisito funcional 8: Funcionalidades para clientes vía web.**
 - Crear cuenta e iniciar sesión.
 - Buscar e inscribirse en cursos.
 - Consultar progreso.
 - Actualizar perfil y detalles de pago.
 - Solicitar soporte técnico.
 - Dejar reseñas y calificaciones.
 - Aplicar cupones de descuento.

3.3 Requisitos no funcionales

3.3.1 Requisitos de rendimiento

- El sistema debe permitir que al menos 100 personas lo usen al mismo tiempo sin problemas.
- El 95% de las funciones deben responder en menos de 2 segundos.

3.3.2 Seguridad

- Los usuarios deben iniciar sesión con autenticación segura.
- Se deben guardar registros de las actividades importantes.
- Toda la comunicación debe estar cifrada (por ejemplo, usando HTTPS).

3.3.3 Fiabilidad

- El sistema debe funcionar correctamente en al menos 999 de cada 1.000 operaciones.

3.3.4 Disponibilidad

- El sistema debe estar disponible el 98% del tiempo cada mes.

3.3.5 Mantenibilidad

- Se debe realizar mantenimiento preventivo regularmente para evitar fallos futuros.
- Las tareas incluyen: actualización del software, limpieza de archivos temporales, verificación de la integridad de la base de datos, y escaneo del sistema contra virus y malware.
- También debe haber documentación clara que facilite los arreglos o mejoras necesarias.
- Se deben hacer respaldos de la información una vez por semana como parte de las tareas preventivas.

3.3.6 Portabilidad

- El sistema debe funcionar tanto en Windows como en Linux.
- Se deben usar tecnologías de código abierto que ya conocen, como Java, Python, MySQL, HTML5, CSS, JSON, SQL Developer y DataModeler.

3.4 Otros Requisitos

- Soporte multilinguaje: español e inglés.
- Accesibilidad para personas con discapacidad visual.

4 Plan de migración:

Fase 1: Análisis y Diseño

- Hallar los partes del sistema monolítico que se volverán en microservicios.
- Diseñar la arquitectura de microservicios y establecer las conexiones entre ellos.

Fase 2: Implementación

- Hacer los microservicios poco a poco.
- Poner pruebas unidad y integración por cada micronervicio.

Fase 3: Despliegue

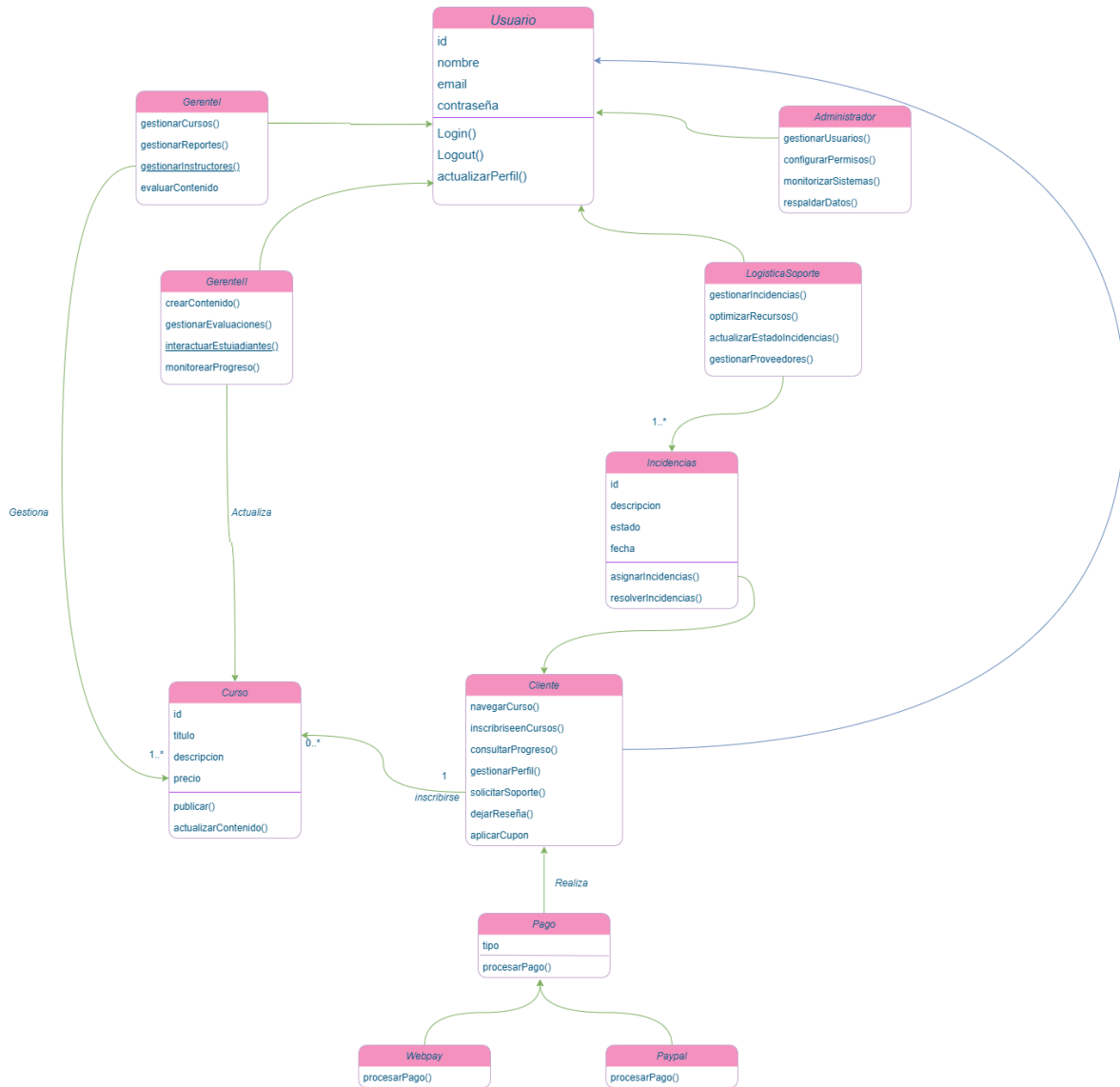
- Poner los micronervicios en la nube usando Kubernetes.
- Ajustar el API Gateway y balancear la carga

Fase 4: Chequear y Mejorar

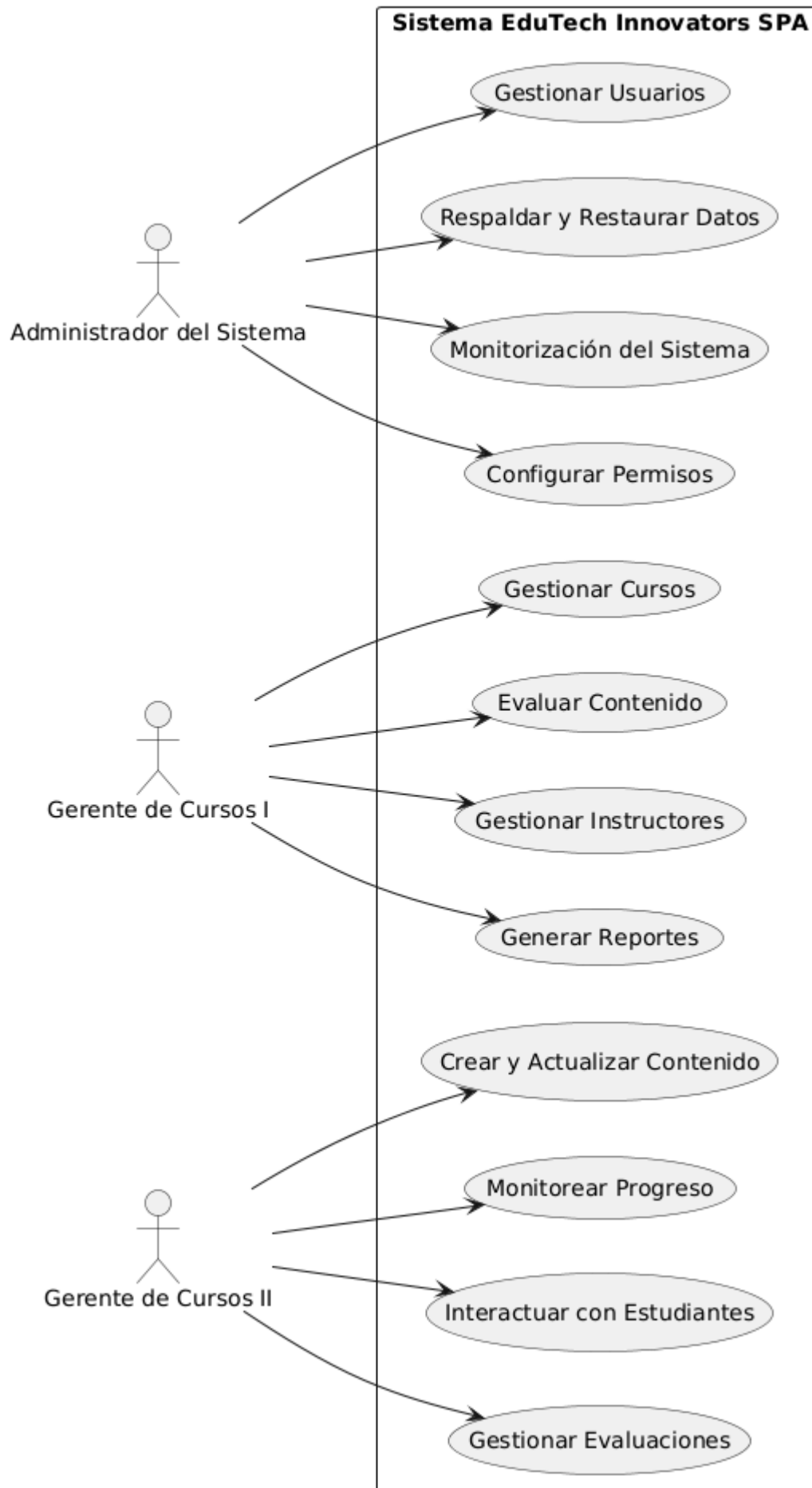
- Ver como funcionan los microservicios y hacer cambios si es necesario.
- Hacer que la base de datos y el almacenamiento de datos del usuario funcionen mejor para aumentar el rendimiento.

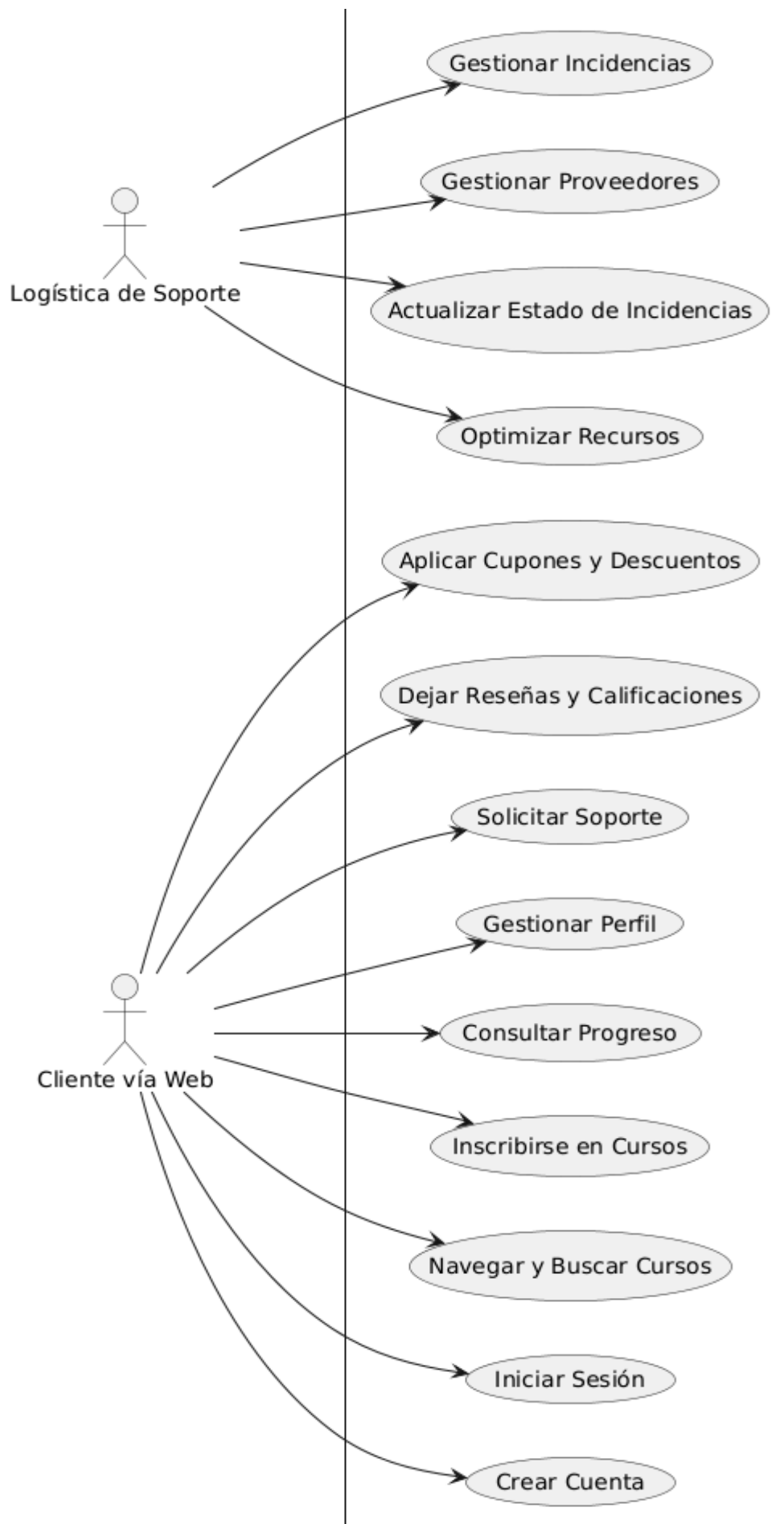
5 Diagramas

5.1 Diagrama de Clase



5.2 Diagrama de clase de uso





5.3 Diagrama de arquitectura

